

Deep Sequence Modeling

Recurrent networks, LSTMs, and the road to attention

● Lecture 2

Prathyusha Dokku



Where does the ball go next?



no history

Any answer you give is a guess.

And now?



to the right

Same ball. The only thing that changed is that you can see its past.

**For sequential data,
the past is the information.**



Sequential data is everywhere

Anything where order matters and earlier things shape later ones



Speech



Text



ECG signals



Stock prices



DNA



Weather



Video



Proteins

Four shapes of sequence problem

One to one



network



Will I pass?

Many to one



network



Sentiment

One to many



network



Caption an image

Many to many



network



Translation

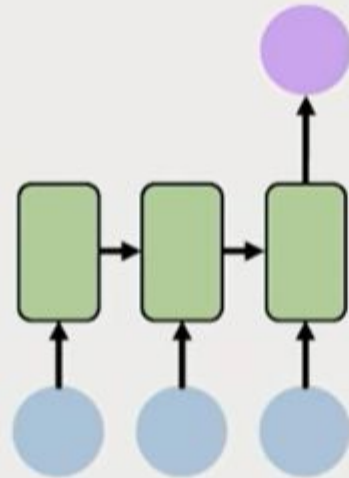
Sequence Modeling Applications



One to One
Binary Classification



"Will I pass this class?"
Student → Pass?

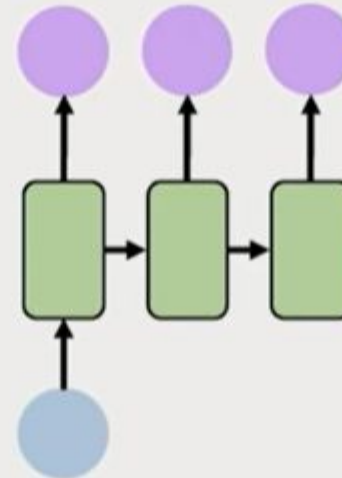


Many to One
Sentiment Classification



The @MIT Introduction to #DeepLearning is definitely one of the best courses of its kind currently available online
introtodeeplearning.com

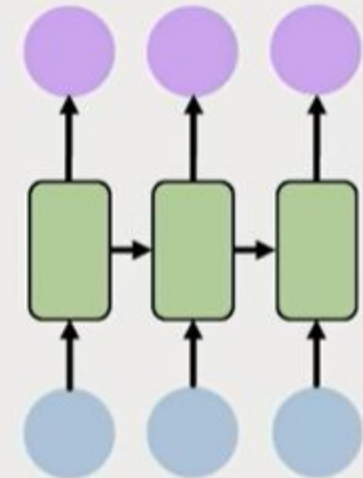
12:45 PM · 12 Feb 2018



One to Many
Image Captioning



"A baseball player throws a ball."



Many to Many
Machine Translation



What a good sequence model must do

We will score every architecture against this

1

Handle variable length

a sentence is 3 words or 300

2

Track long dependencies

word 1 can decide word 40

3

Preserve order

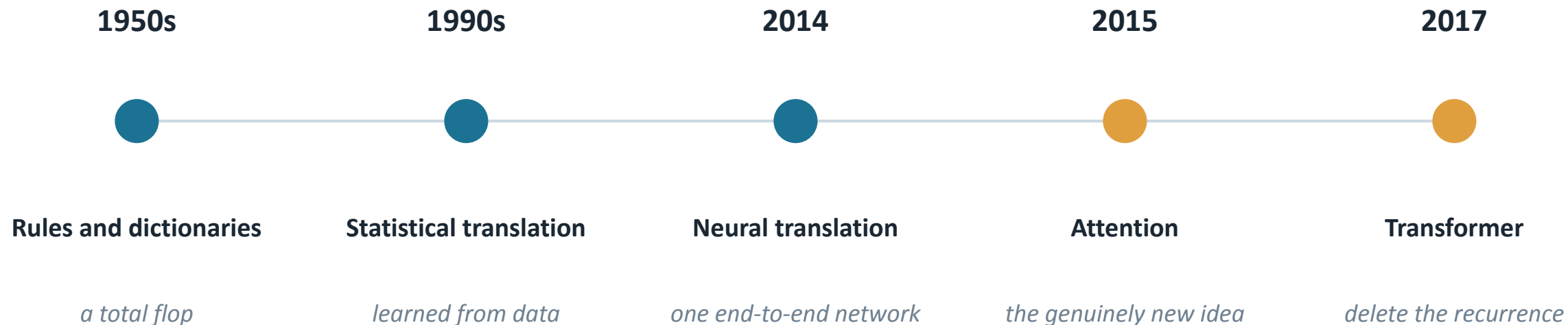
"dog bites man" is not "man bites dog"

4

Share parameters

no separate model for word #7

A very short history



Machine translation is where NLP began, and it drove nearly every idea we still use.

**Almost everything in deep learning
was invented last millennium.**

Attention is the rare exception.



Part 1

Words into numbers

Before any model, a problem nobody can skip



Our running example

"This morning I took my cat for a ____"

Task: predict the next word.

It looks like a toy problem. Every large language model is trained on exactly this.

So how does a network read this?

**How does a neural network
read the word "cat"?**

ask the room

It cannot.

Neural networks operate on numbers.



Embedding: three steps

1 Build a vocabulary

```
a  
cat  
dog  
walk
```

2 Map to an index

```
a      -> 1  
cat    -> 2  
dog    -> 3  
walk   -> 4
```

3 Index to a vector

```
cat ->  
[0.21, -0.44,  
 0.87, ... ]
```

An embedding is a lookup: index in, fixed-size vector out.

Option A: one-hot

cat $\rightarrow$ [0, 1, 0, 0, 0, ... , 0]

Honest and simple

One slot per word. You can always read the word back out.

But fatally flawed

Every word is equally far from every other word.

"cat" is exactly as similar to "dog" as it is to "bulldozer"

Option B: a learned embedding

cat $\rightarrow$ [0.21, -0.44, 0.87, 0.05, ...]

similar words land near each other

cat
dog
horse

red
blue

ran
jumped

Dense, lower-dimensional, and trained along with everything else.

Part 2

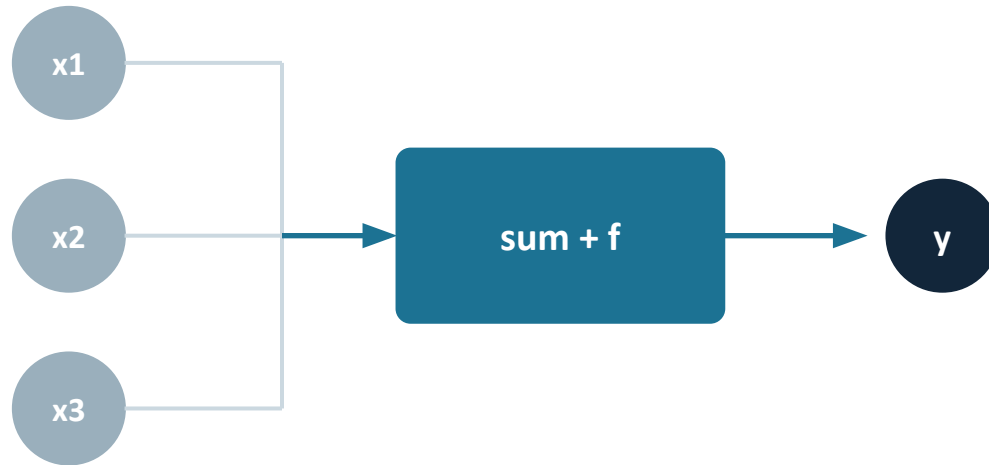
Building an RNN

We are going to derive it, not be shown it



Start where we left off

Inputs in, weighted sum, nonlinearity, output

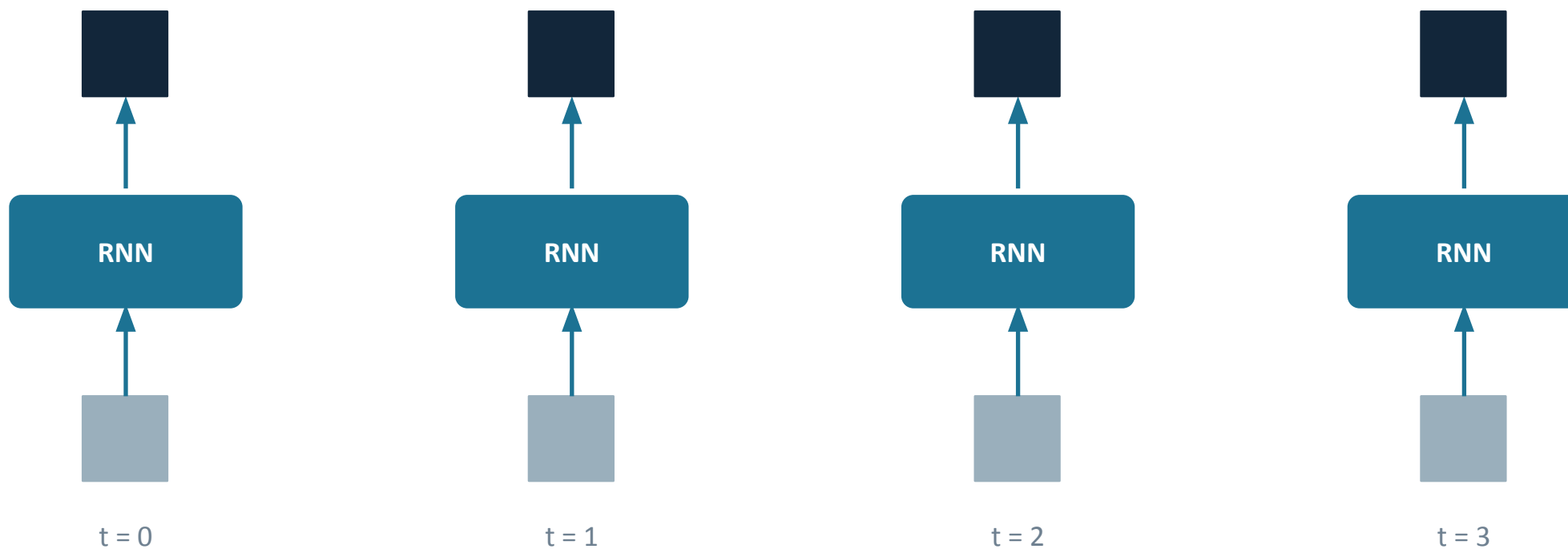


Notice

These inputs are all one slice.
One moment.

There is no time here.

Attempt 1: repeat it at every time step



four copies of the same network, side by side

Attempt 1

What is wrong with this?

ask the room

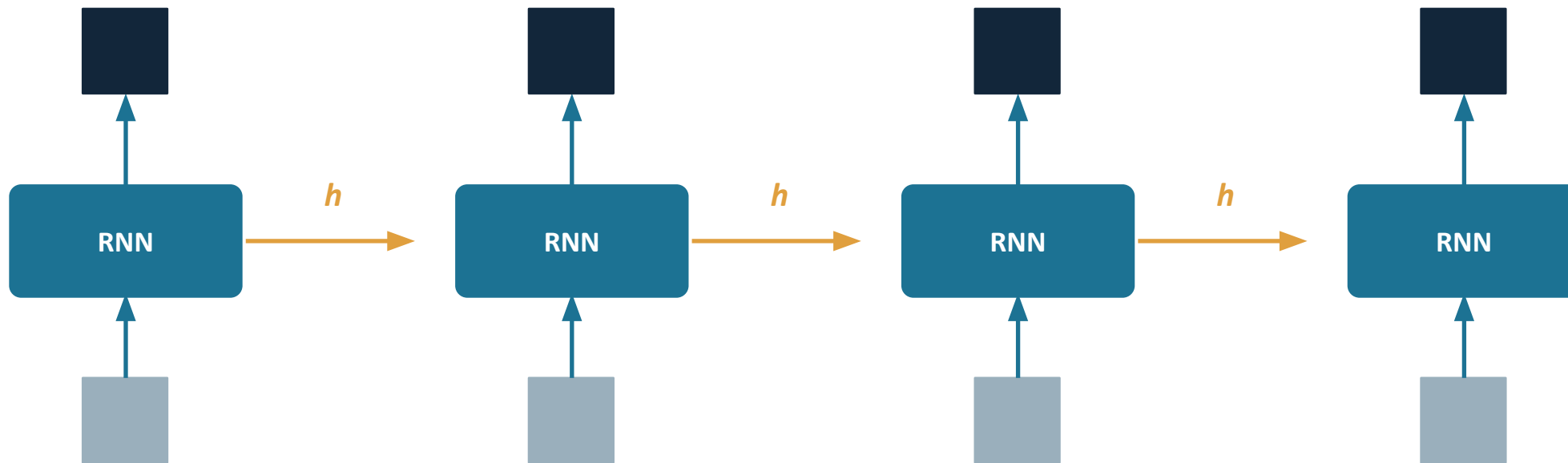
Every copy works in isolation.

We just deleted the history.



Attempt 2: pass a state forward

Let each step hand a memory to the next one



h carries memory. The prediction now depends on this input AND everything before it.

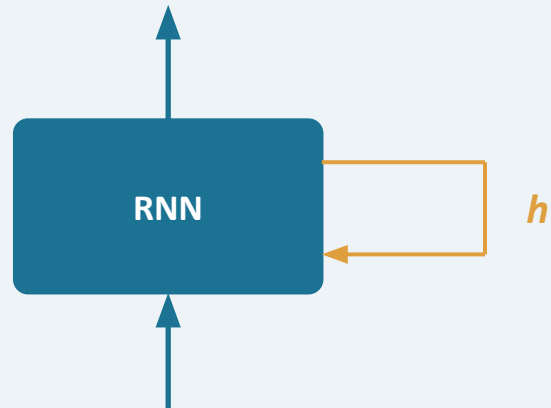
**State passed forward and updated
is called a recurrence relation.**

This is a recurrent neural network.



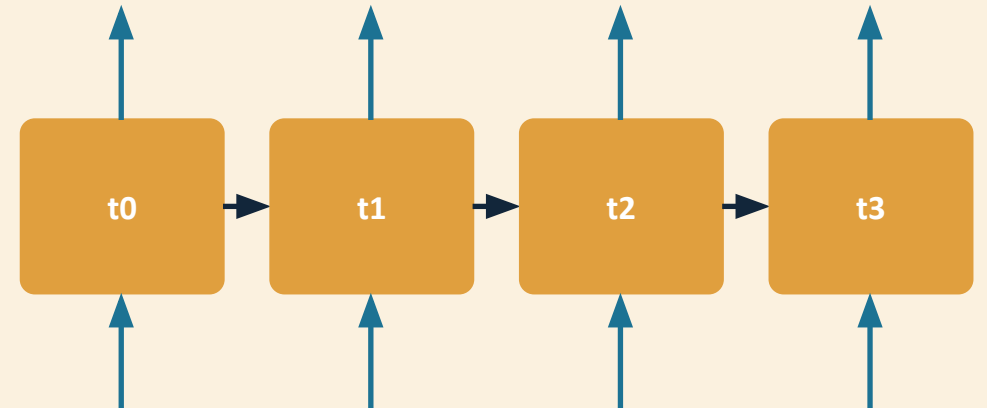
Two views of the same network

Compact



state loops back into itself

Unrolled across time



same weights reused at every step

The two equations

$$\begin{aligned} h(t) &= \tanh (W_{hh} \cdot h(t-1) + W_{xh} \cdot \\ x(t)) \end{aligned}$$

update the state

$$y(t) = W_{hy} \cdot h(t)$$

produce the output

W_{xh} input into state

W_{hh} state into state

W_{hy} state into output

**The same three matrices
at every single time step.**

Reused, not duplicated.



An RNN in five lines

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

for word in ["I", "love", "recurrent", "neural"]:
    prediction, hidden_state = my_rnn(word,
    hidden_state)

print(prediction)      # >>> "networks"
```

The loop is the model

word in, state in

prediction out, state out

state goes straight back in

The same thing, in real code

You would use SimpleRNN or torch.nn.RNN in practice

```
class MyRNNCell(tf.keras.layers.Layer):  
    def __init__(self, units, in_dim, out_dim):  
        self.W_xh = self.add_weight([units, in_dim])  
        self.W_hh = self.add_weight([units, units])  
        self.W_hy = self.add_weight([out_dim, units])  
        self.h = tf.zeros([units, 1])  
  
    def call(self, x):  
        self.h = tf.tanh(self.W_hh @ self.h + self.W_xh @ x)  
        return self.W_hy @ self.h, self.h
```

Scorecard: how does the RNN do?

Requirement	RNN
Variable length	yes
Long dependencies	in theory...
Order preserved	yes
Parameter sharing	yes